

# CSS

## Estilos e Layout

Da estrutura sem cor da Aula 2 a uma interface com identidade visual, layout organizado e responsividade de verdade.

---

### DISCIPLINA

**Programação — Web Coding (CC/P2)**

### MINISTRADO POR

**Prof. Esp. Daniel Costa**

estilos.css

```
body {  
  font-family: "Segoe UI", sans-serif;  
  background: #f2f0f8;  
}  
  
.cartao {  
  display: flex;  
  gap: 16px;  
  padding: 24px;  
}
```

# O que vamos construir em Web Coding

Seis blocos, na ordem em que a própria Web é montada. Hoje avançamos do bloco 02 para o bloco 03.

01

## Web

Como a Internet e as aplicações Web funcionam.

02

## HTML5

A estrutura. O esqueleto de qualquer página.

03

## CSS

A aparência. Cores, layout, responsividade — hoje.

04

## JavaScript

O comportamento. Lógica, interação, DOM.

05

## Bootstrap

Agilidade. Componentes prontos e responsivos.

06

## JSON + SQLite

Os dados. Armazenar, consultar, persistir.

**Hoje:** estamos no bloco 03 — CSS. A estrutura de HTML da Aula 2 continua exatamente a mesma; o que muda é como ela aparece na tela.

# De onde partimos hoje

A Página de Perfil Pessoal da Aula 2 já está pronta — hoje ela ganha aparência.

## O QUE JÁ TEMOS

### HTML5 semântico e completo

- header, main com sections, footer.
- Texto, lista, tabela, formulário — tudo funcional.
- Zero CSS até agora: só estrutura.

## O QUE MUDA HOJE

### A mesma página, com identidade visual

- Cores, tipografia e espaçamento (Box Model).
- Layout organizado com Flexbox e Grid.
- Responsividade: a página se adapta a qualquer tela.

Nenhuma tag HTML nova entra hoje. Vamos apenas **ligar um arquivo CSS** à mesma página e transformá-la.

# O que é CSS?

CSS = **C**ascading **S**tyle **S**heets — folhas de estilo em cascata. Ele separa o conteúdo (HTML) da aparência (CSS).

## SEPARAÇÃO DE PAPÉIS

### Conteúdo x aparência

- HTML diz **o que** existe na página.
- CSS diz **como** aquilo aparece: cor, tamanho, posição.
- Trocar o CSS não muda o conteúdo — só a aparência.

## CASCATA

### De onde vem o "C" de CSS

- Várias regras podem mirar o mesmo elemento.
- Quando há conflito, vence a regra mais específica (ou a última, em empate).
- Por isso a ordem e a especificidade do seletor importam.

A mesma estrutura HTML pode virar sites completamente diferentes, só trocando o CSS.

## \$ CONCEITO

# Como ligar o CSS ao HTML

Três formas existem — só uma é recomendada no dia a dia.

```
index.html

<head>
  <link rel="stylesheet" href="estilos.css">
</head>
```

| FORMA   | COMO FUNCIONA                                                                                  |
|---------|------------------------------------------------------------------------------------------------|
| Externa | Arquivo <code>.css</code> separado, ligado com <code>&lt;link&gt;</code> . <b>Recomendada.</b> |
| Interna | Bloco <code>&lt;style&gt;</code> dentro do <code>&lt;head&gt;</code> .                         |
| Inline  | Atributo <code>style=""</code> direto na tag. Evitar — difícil de manter.                      |

Arquivo externo mantém HTML e CSS separados e permite reaproveitar o mesmo estilo em várias páginas.

# A anatomia de uma regra CSS

Toda regra CSS segue o mesmo formato: um seletor e um bloco de declarações entre chaves.

```
anatomia

h1 {
  color: #2e2b45;
  font-size: 28px;
}

seletor  propriedade  valor
```

**seletor** Escolhe **quais** elementos recebem o estilo.

**propriedade** O que será alterado: `color`, `font-size`, `margin`...

**valor** Como fica a propriedade. Cada declaração termina com `;`.

# Seletores básicos

Três formas de escolher elementos: por tag, por classe ou por identificador.

```
seletores.css

p { color: #4a4368; }

.destaque { font-weight: bold; }

#cabecalho { background: #1b1730; }
```

| SELETOR    | ESCOLHE                                               | USO NO HTML      |
|------------|-------------------------------------------------------|------------------|
| p          | Toda tag <p> da página.                               | —                |
| .destaque  | Qualquer elemento com class="destaque". Pode repetir. | class="destaque" |
| #cabecalho | O único elemento com id="cabecalho". Não repete.      | id="cabecalho"   |

# Combinando e agrupando seletores

Seletores podem ser combinados para mirar exatamente o elemento certo.

combinando.css

```
main p { line-height: 1.6; }
```

```
h1, h2, h3 { font-family: "Segoe UI"; }
```

```
a:hover { color: #8b6fd6; }
```

**main p** **Descendente:** todo p dentro de main (espaço entre seletores).

**h1, h2, h3** **Agrupamento:** a vírgula aplica a mesma regra a vários seletores.

**a:hover** **Pseudo-classe:** estilo aplicado só num estado, como o mouse sobre o link.



# Cores

Quatro formas de escrever uma cor em CSS — todas produzem o mesmo resultado.

```
cores.css

color: purple;           /* nome */
color: #8b6fd6;          /* hexadecimal */
color: rgb(139, 111, 214); /* vermelho, verde, azul */
color: rgba(139, 111, 214, 0.5); /* com transparência */
```

**color** pinta o texto; **background-color** (ou **background**) pinta o fundo do elemento.

rgba e hsl permitem transparência (o último valor, **alpha**, vai de 0 a 1).

# Tipografia

As propriedades que definem como o texto aparece na tela.

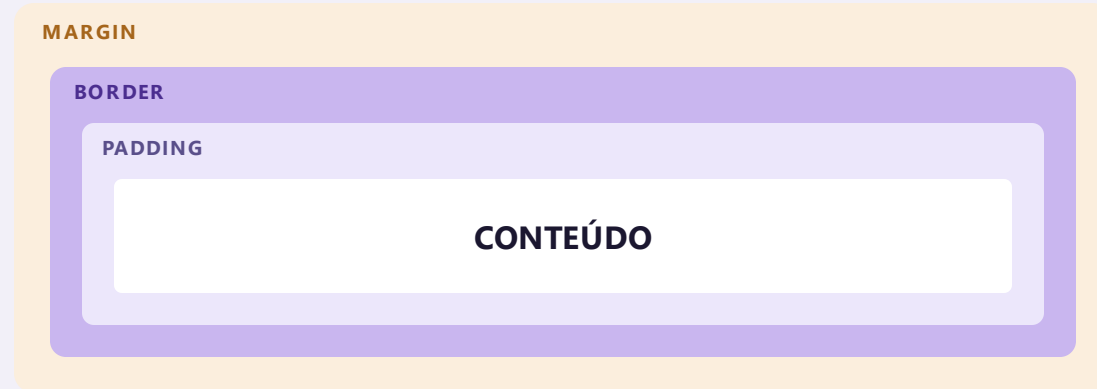
tipografia.css

```
body {  
  font-family: "Segoe UI", sans-serif;  
  font-size: 16px;  
  line-height: 1.5;  
}  
  
h1 {  
  font-weight: 700;  
  text-align: center;  
}
```

| PROPRIEDADE | CONTROLA                             |
|-------------|--------------------------------------|
| font-family | A fonte usada.                       |
| font-size   | O tamanho do texto.                  |
| font-weight | A espessura (normal, bold, 400–900). |
| line-height | O espaço entre linhas.               |
| text-align  | Alinhamento: left, center, right.    |

# Todo elemento é uma caixa

O Box Model descreve as quatro camadas que envolvem o conteúdo de qualquer elemento.



**conteúdo** O texto ou elemento em si.

**padding** Espaço interno, entre o conteúdo e a borda.

**border** A borda visível ao redor do elemento.

**margin** Espaço externo, entre este elemento e os vizinhos.

# box-sizing e dimensões

Uma linha de CSS resolve a maior confusão do Box Model: o tamanho final da caixa.

```
dimensoes.css

* {
  box-sizing: border-box;
}

.cartao {
  width: 100%;
  max-width: 600px;
  padding: 24px;
}
```

| UNIDADE | SIGNIFICA                                         |
|---------|---------------------------------------------------|
| px      | Pixels — valor fixo.                              |
| %       | Porcentagem do elemento pai.                      |
| em      | Relativo ao tamanho de fonte do próprio elemento. |
| rem     | Relativo ao tamanho de fonte da página (root).    |
| vh / vw | Porcentagem da altura/largura da tela.            |

box-sizing: border-box faz padding e border entrarem **dentro** do width — sem ele, eles se somam e "estouram" o layout.

# A propriedade position

Controla como um elemento se posiciona em relação ao fluxo normal da página.

| VALOR                 | COMPORTAMENTO                                                                                                             |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>static</code>   | Padrão. Segue o fluxo normal da página.                                                                                   |
| <code>relative</code> | Fica no fluxo, mas pode ser deslocado com <code>top/left</code> , a partir da própria posição.                            |
| <code>absolute</code> | Sai do fluxo; posiciona em relação ao ancestral mais próximo com <code>position</code> diferente de <code>static</code> . |
| <code>fixed</code>    | Sai do fluxo; fica fixo na tela mesmo com o scroll da página.                                                             |
| <code>sticky</code>   | Segue o fluxo até um ponto, depois "gruda" na tela durante o scroll.                                                      |

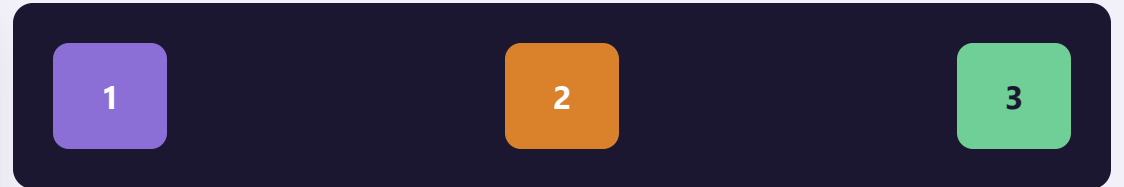
Um cabeçalho que acompanha o scroll usa `position: sticky; top: 0;` — muito comum em sites reais.

# Flexbox: layout em uma direção

Transforma um elemento em um "contêiner flexível" — seus filhos se organizam automaticamente em linha ou coluna.

```
flex.css

.container {
  display: flex;
  gap: 12px;
  justify-content: space-between;
}
```

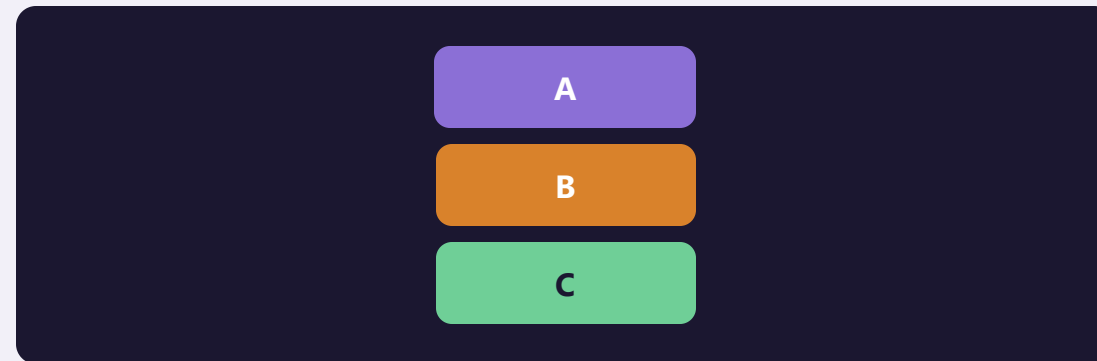


O código à esquerda gera exatamente o resultado à direita — três caixas espaçadas automaticamente, sem margin manual.

# As propriedades que você mais vai usar

Um pequeno grupo de propriedades resolve a maioria dos layouts do dia a dia.

| PROPRIEDADE                  | FAZ                                            |
|------------------------------|------------------------------------------------|
| <code>flex-direction</code>  | row (padrão) ou column.                        |
| <code>justify-content</code> | Alinha no eixo principal (horizontal, em row). |
| <code>align-items</code>     | Alinha no eixo cruzado (vertical, em row).     |
| <code>flex-wrap</code>       | Permite quebrar para a próxima linha.          |
| <code>gap</code>             | Espaço entre os itens, sem precisar de margin. |

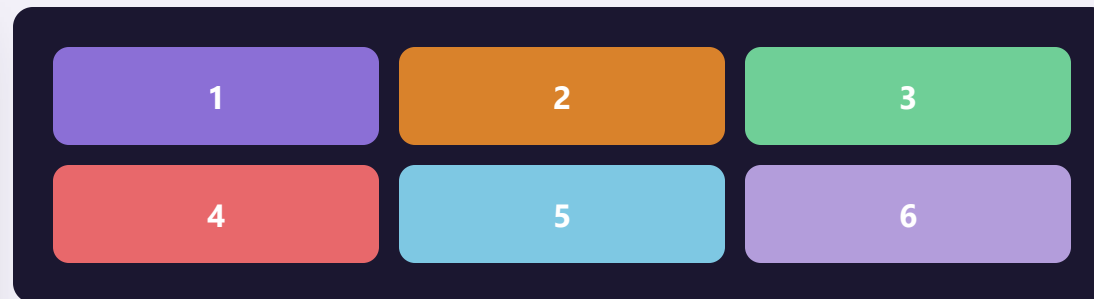


# Grid: layout em duas direções

Enquanto Flexbox pensa em uma linha por vez, Grid organiza linhas e colunas ao mesmo tempo.

```
grid.css

.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```



fr significa "fração do espaço disponível" — 1fr 1fr 1fr cria três colunas de larguras iguais.



# Quando usar Grid ou Flexbox

As duas ferramentas se complementam — não competem.

## USE FLEXBOX QUANDO

### Uma direção só

- Uma barra de navegação, uma linha de botões.
- Centralizar conteúdo dentro de uma caixa.
- O tamanho dos itens pode variar naturalmente.

## USE GRID QUANDO

### Linhas e colunas juntas

- O layout geral da página (cabeçalho, menu, conteúdo, rodapé).
- Galerias de cartões alinhados em grade.
- Você já sabe quantas colunas quer.

É comum usar Grid para o layout geral da página e Flexbox dentro de cada bloco — as duas técnicas convivem bem.

# Media queries

Blocos de CSS que só valem a partir (ou até) de um determinado tamanho de tela.

```
responsivo.css

.container {
  display: grid;
  grid-template-columns: 1fr 1fr;
}

@media (max-width: 600px) {
  .container {
    grid-template-columns: 1fr;
  }
}
```

**Breakpoint** é o tamanho de tela onde o layout muda. 600px, 768px e 1024px são pontos comuns para celular, tablet e desktop.

# Mobile-first e a tag viewport

Uma linha no HTML e uma decisão de abordagem — as duas bases de qualquer site responsivo.

```
index.html  
  
<meta name="viewport"  
      content="width=device-width, initial-scale=1">
```

**viewport** Sem essa tag, o celular renderiza como se fosse desktop e depois encolhe — texto minúsculo.

**mobile-first** Escrever o CSS pensando no celular primeiro, e usar media queries para *adicionar* estilo em telas maiores.

A tag viewport vai no <head> do HTML — não é CSS, mas é indispensável para qualquer responsividade funcionar.

# Antes do projeto: quatro desafios rápidos

Crie um arquivo `pratica.css` ligado a uma página simples e resolva em sequência.

- 01 Estilize um **h1**: cor, tamanho de fonte e alinhamento centralizado.
- 02 Dê padding, border-radius e uma cor de fundo a um **div** com `class="cartao"`.
- 03 Transforme uma lista em uma linha de itens usando **display: flex** e **gap**.
- 04 Adicione uma **media query** que muda a cor de fundo da página abaixo de 600px.

Terminou os quatro? Essas técnicas são exatamente as que vamos aplicar na Página de Perfil Pessoal agora.

# Estilizando a Página de Perfil Pessoal

O mesmo HTML da Aula 2 ganha um arquivo `estilos.css` novo — nenhuma tag muda.

## HEADER

### Cabeçalho

Flexbox para alinhar foto e nome, cor de fundo e tipografia.

## SECTION

### Cartões

Box Model: padding, border-radius e sombra em cada seção.

## UL

### Habilidades

Flexbox para exibir as habilidades como "tags" lado a lado.

## FORM

### Contato

Grid ou Flexbox para organizar os campos do formulário.

## @MEDIA

### Responsivo

Layout se ajusta em telas menores que 600px.

# estilos.css de partida

Um ponto de partida — combine e ajuste os valores para deixar a página com a sua cara.

estilos.css

```
* { box-sizing: border-box; }

body {
  font-family: "Segoe UI", sans-serif;
  background: #f2f0f8;
  margin: 0;
}

header {
  display: flex;
  align-items: center;
  gap: 20px;
  background: #1b1730;
  color: #fff;
  padding: 24px;
}

main section {
  background: #fff;
  border-radius: 10px;
  padding: 20px;
  margin: 16px;
```

# Checklist de entrega

Antes de considerar pronto, confira se o seu CSS cobre todos os itens abaixo.

✓ Arquivo estilos.css criado e ligado com <link> no head

✓ Cores e tipografia aplicadas ao body e aos títulos

✓ Cada section com padding, border-radius (Box Model)

✓ header organizado com Flexbox

✓ Lista de habilidades organizada com Flexbox ou Grid

✓ Uma media query ajustando o layout em telas pequenas

Próxima aula, 14/09: JavaScript — a página ganha comportamento além da aparência.

# Propriedades vistas hoje

Uma colinha rápida com o vocabulário CSS desta aula.

| PROPRIEDADE                                | CATEGORIA      | FUNÇÃO                                         |
|--------------------------------------------|----------------|------------------------------------------------|
| <code>color / background</code>            | Cores          | Cor do texto e do fundo                        |
| <code>font-family / font-size</code>       | Tipografia     | Fonte usada e tamanho do texto                 |
| <code>padding / margin / border</code>     | Box Model      | Espaço interno, externo e borda                |
| <code>box-sizing</code>                    | Box Model      | Inclui padding/border no width                 |
| <code>width / max-width</code>             | Dimensões      | Largura do elemento                            |
| <code>position</code>                      | Posicionamento | static, relative, absolute, fixed, sticky      |
| <code>display: flex</code>                 | Flexbox        | Layout em uma direção                          |
| <code>justify-content / align-items</code> | Flexbox        | Alinhamento dos itens flex                     |
| <code>display: grid</code>                 | Grid           | Layout em linhas e colunas                     |
| <code>grid-template-columns</code>         | Grid           | Define as colunas do grid                      |
| <code>@media</code>                        | Responsividade | Regras que valem só em certos tamanhos de tela |



PARA LEVAR DAQUI

# Quatro ideias que resolvem o resto

01

## HTML estrutura, CSS estiliza

A mesma página pode virar sites completamente diferentes só trocando o CSS.

03

## Flexbox para uma direção, Grid para duas

As duas técnicas se complementam — use as duas juntas.

02

## box-sizing: border-box evita surpresas

Padding e border passam a entrar dentro do width.

04

## Responsivo começa pelo celular

Mobile-first + media queries adaptam o layout a qualquer tela.

### FIM DA AULA 3

# Obrigado.

A página que era só estrutura agora tem cor, layout e responde ao tamanho da tela.  
Na próxima aula, 14/09, ela ganha comportamento com **JavaScript**.

---

DÚVIDAS SÃO BEM-VINDAS

**Prof. Esp. Daniel Costa**